System Tasks in Verilog HDL

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **3**

2 Basic Syntax **3**

3 The `$display` Task **4**
3.1 Syntax 4
3.2 Example 4

4 The `$write` Task **5**
4.1 Example 5

5 Format Specifiers **6**
5.1 Example 6

6 The `$monitor` Task **7**
6.1 Example 7
6.2 Important Points 7

7 The `$monitoron` and `$monitoroff` Tasks **8**
7.1 Example 8

8 The `$strobe` Task **8**
8.1 Example 8

9 The `$time`, `$stime`, and `$realtime` Tasks **9**
9.1 Example 9

10 The `$finish` Task **10**
10.1 Example 10

11 The `$stop` Task **10**
11.1 Example 10

12 The `$random` Task **11**
12.1 Example 11
12.2 The `$urandom` Function 11

13 Waveform Dumping Tasks **12**

13.1 The `$dumpfile` Task 12
13.2 The `$dumpvars` Task 12
13.3 Complete Example 12

14 The `$readmemh` and `$readmemb` Tasks **13**
14.1 Example: Hexadecimal Memory File 13
14.2 Example: Binary Memory File 14

15 File I/O System Tasks **14**
15.1 Example 14

16 The `$test$plusargs` and `$value$plusargs` Functions **15**
16.1 Example 15

17 The `$signed` and `$unsigned` Functions **16**
17.1 Example 16

18 The `$clog2` Function **16**
18.1 Example 16

19 System Tasks vs. User-Defined Tasks **17**

20 Common Errors and Best Practices **17**

21 System Tasks Summary **18**

22 Summary **19**

1 Introduction

System tasks are predefined operations provided by Verilog HDL for simulation, debugging, displaying information, controlling simulation, generating waveform files, and performing other verification-related activities.

System tasks begin with the dollar sign (`$`).

Unlike ordinary user-defined tasks, system tasks are built into the Verilog language or supported by the simulation environment.

Common system tasks include:

- `$display`
- `$write`
- `$monitor`
- `$strobe`
- `$time`
- `$finish`
- `$stop`
- `$random`
- `$dumpfile`
- `$dumpvars`
- `$readmemh`
- `$readmemb`

System tasks are primarily used in testbenches and simulation environments. Many are not synthesizable into hardware.

2 Basic Syntax

A system task is called by writing its name with a dollar sign, followed by parentheses when required.

```
1 $display("Hello, Verilog!");
2
3 $finish;
```

Some system tasks accept arguments, while others can be called without parentheses.

The arguments may include strings, signal values, file handles, or simulation control parameters.

3 The $display Task

The $display task prints formatted information to the simulator's output console.

It automatically adds a newline at the end of the displayed message.

3.1 Syntax

```
$display("format_string", arguments);
```

3.2 Example

```
module display_example;

    reg a, b;
    wire y;

    assign y = a & b;

    initial begin
        a = 0;
        b = 0;
        #5;

        $display("a=%b b=%b y=%b", a, b, y);

        a = 1;
        b = 1;
        #5;

        $display("a=%b b=%b y=%b", a, b, y);
        $finish;
    end

endmodule
```

Output:

```
a=0 b=0 y=0
a=1 b=1 y=1
```

The task is useful for printing signal values at a specific point during simulation.

4 The $write Task

The $write task prints formatted information without automatically adding a newline at the end.

4.1 Example

```
module write_example;

    initial begin
        $write("Hello ");
        $write("Verilog ");
        $write("HDL!");
        $display("");
        $finish;
    end

endmodule
```

Output:

```
Hello Verilog HDL!
```

The $write task is useful when multiple messages must appear on the same line.

5 Format Specifiers

Format specifiers determine how values are displayed by system tasks.

Specifier	Meaning
%b	Binary representation
%d	Decimal representation
%h	Hexadecimal representation
%o	Octal representation
%c	Character representation
%s	String representation
%t	Time representation
%m	Current hierarchical instance name
%v	Strength information for supported net values
%%	Prints a percent sign

5.1 Example

```
module format_example;

    reg [7:0] data;

    initial begin
        data = 8'd25;

        $display("Binary      = %b", data);
        $display("Decimal     = %d", data);
        $display("Hexadecimal = %h", data);
        $display("Octal       = %o", data);

        $finish;
    end

endmodule
```

Output:

```
Binary      = 00011001
Decimal     =  25
Hexadecimal = 19
Octal       = 031
```

The exact spacing and leading zeros can depend on the format string and simulator formatting.

6 The $monitor Task

The $monitor task prints the specified values whenever one or more monitored arguments change.

A new monitoring statement replaces the previous active monitoring statement in the same simulation.

6.1 Example

```
module monitor_example;

    reg a, b;
    wire y;

    assign y = a & b;

    initial begin
        $monitor("Time=%0t a=%b b=%b y=%b",
                 $time, a, b, y);

        a = 0;
        b = 0;

        #10 a = 1;
        #10 b = 1;
        #10 a = 0;

        #10 $finish;
    end

endmodule
```

The monitor prints the signal values when the monitored values change, including the initial scheduled observation.

6.2 Important Points

- Monitoring remains active until replaced or disabled.
- Only one active $monitor statement exists at a time.
- It is useful for observing signal changes without writing repeated display statements.

7 The $monitoron and $monitoroff Tasks

These tasks enable or disable output from the active `$monitor` statement.

7.1 Example

```
module monitor_control_example;

    reg count;

    initial begin
        $monitor("Time=%0t count=%b", $time, count);

        count = 0;
        #5 count = 1;

        $monitoroff;
        #5 count = 0;

        $monitoron;
        #5 count = 1;

        #5 $finish;
    end

endmodule
```

When monitoring is disabled, changes do not produce monitor output. Monitoring can be enabled again using `$monitoron`.

8 The $strobe Task

The `$strobe` task prints values at the end of the current simulation time step, after the current time-step's updates have settled.

This can help observe final values when multiple updates occur at the same simulation time.

8.1 Example

```
module strobe_example;
    reg a;
    initial begin
        a = 0;
        a = 1;
        $display("Display value = %b", a);
        $strobe("Strobe value  = %b", a);
        #1 $finish;
    end
endmodule
```

The exact result depends on the simulation event scheduling and the order of updates. In this example, both statements generally observe `a = 1` because the blocking assignment updates `a` immediately.

`$strobe` is especially useful when nonblocking assignments or other scheduled updates must complete before values are printed.

9 The `$time`, `$stime`, and `$realtime` Tasks

These system functions return simulation time in different forms.

- `$time` returns simulation time as an integer time value.
- `$stime` returns simulation time as a 32-bit unsigned integer in traditional Verilog.
- `$realtime` returns simulation time as a real number.

9.1 Example

```
1  `timescale 1ns/1ps
2
3  module time_example;
4
5      initial begin
6          #2.5;
7
8          $display("Time       = %0t", $time);
9          $display("Real time  = %0.3f", $realtime);
10
11         #2.5;
12         $display("Final time = %0t", $time);
13
14         $finish;
15     end
16
17 endmodule
```

The time unit and precision affect how delays are represented and rounded. The displayed value of `$time` may be rounded to an integer time unit.

Use `$realtime` when fractional time values are important.

10 The $finish Task

The $finish task terminates the simulation.

10.1 Example

```
module finish_example;

    initial begin
        $display("Simulation started");

        #10;
        $display("Simulation completed");

        $finish;
    end

endmodule
```

The simulation ends when $finish executes.

Some simulators accept an optional numeric argument that controls the amount of diagnostic information printed when simulation terminates.

11 The $stop Task

The $stop task suspends or interrupts simulation so that the user can inspect the current simulation state.

Its behavior depends partly on the simulator and its interactive debugging configuration.

11.1 Example

```
module stop_example;
    reg clk;
    initial begin
        clk = 0;

        #10 clk = 1;
        $display("Simulation paused");
        $stop;
        #10 clk = 0;
        $finish;
    end
endmodule
```

Unlike $finish, which terminates simulation, $stop is mainly useful during interactive debugging.

12 The $random Task

The $random system function generates a pseudo-random signed integer.

It is useful for creating test values in simulation testbenches.

12.1 Example

```
module random_example;

    integer value;

    initial begin
        repeat (5) begin
            value = $random;
            $display("Random value = %0d", value);
        end

        $finish;
    end

endmodule
```

The generated sequence depends on the simulator and the random seed.

12.2 The $urandom Function

In SystemVerilog, $urandom generates a 32-bit unsigned pseudo-random value.

```
module urandom_example;

    integer i;
    reg [31:0] value;

    initial begin
        for (i = 0; i < 5; i = i + 1) begin
            value = $urandom;
            $display("Random value = %0d", value);
        end

        $finish;
    end

endmodule
```

$urandom is a SystemVerilog feature. Check language mode and simulator support when using it in a Verilog-only environment.

13 Waveform Dumping Tasks

Waveform dumping tasks create files containing signal activity for viewing in a waveform viewer.

Common tasks include:

- `$dumpfile`
- `$dumpvars`
- `$dumpall`
- `$dumpon`
- `$dumpoff`

These tasks are commonly supported by event-driven simulators such as Icarus Verilog.

13.1 The `$dumpfile` Task

Specifies the name of the waveform dump file.

```
$dumpfile("wave.vcd");
```

13.2 The `$dumpvars` Task

Specifies which signals or hierarchy levels should be recorded.

```
$dumpvars(0, testbench);
```

The depth argument `0` requests dumping the selected hierarchy and its descendants.

13.3 Complete Example

```
module waveform_example;

    reg a, b;
    wire y;

    assign y = a & b;
    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, waveform_example);

        a = 0;
        b = 0;

        #10 a = 1;
        #10 b = 1;
        #10 a = 0;

        #10 $finish;
    end
```

```
endmodule
```

The simulator generates a file named `wave.vcd`, which can be opened in a compatible waveform viewer.

These waveform tasks are simulator-oriented and are not synthesizable hardware constructs.

14 The `$readmemh` and `$readmemb` Tasks

These system tasks load memory initialization data from a text file.

- `$readmemh` reads hexadecimal values.
- `$readmemb` reads binary values.

14.1 Example: Hexadecimal Memory File

Suppose `memory.hex` contains:

```
12
34
A5
FF
```

The following code loads the values into an array:

```
module readmemh_example;

    reg [7:0] memory [0:3];
    integer i;

    initial begin
        $readmemh("memory.hex", memory);

        for (i = 0; i < 4; i = i + 1)
            $display("memory[%0d] = %h", i, memory[i]);

        $finish;
    end

endmodule
```

14.2 Example: Binary Memory File

Suppose `memory.bin` contains:

```
00010010
00110100
10100101
11111111
```

Use the following statement:

```
$readmemb("memory.bin", memory);
```

These tasks are useful for initializing memories in simulation and, in supported synthesis flows, for memory initialization.

15 File I/O System Tasks

Verilog provides system tasks and functions for writing information to files.

Common file operations include:

- `$fopen`: Opens a file.
- `$fclose`: Closes a file.
- `$fdisplay`: Writes formatted data to a file and adds a newline.
- `$fwrite`: Writes formatted data without automatically adding a newline.
- `$fmonitor`: Monitors values and writes changes to a file.
- `$fstrobe`: Writes values at the end of the current time step.

15.1 Example

```
module file_example;
    integer file;
    reg [7:0] data;
    initial begin
        file = $fopen("output.txt", "w");
        if (file == 0) begin
            $display("Error opening file");
            $finish;
        end
        data = 8'hA5;
        $fdisplay(file, "Data = %h", data);
        $fwrite(file, "Decimal = %0d\n", data);
        $fclose(file);
        $display("File writing completed");
        $finish;
    end
endmodule
```

The file handle returned by `$fopen` is used by the file output tasks.

Check the file-opening modes supported by your simulator.

16 The $test$plusargs and $value$plusargs Functions

These functions read command-line plus arguments provided when a simulation is launched.

- `$test$plusargs` checks whether a matching argument is present.
- `$value$plusargs` reads a value from a matching argument.

16.1 Example

```
module plusargs_example;

    integer seed;
    reg debug_mode;

    initial begin
        debug_mode = $test$plusargs("DEBUG");

        if ($value$plusargs("SEED=%d", seed))
            $display("Seed = %0d", seed);
        else
            $display("No seed provided");

        if (debug_mode)
            $display("Debug mode enabled");

        $finish;
    end

endmodule
```

Example simulator command:

```
vvp simulation.vvp +DEBUG +SEED=123
```

The exact command-line syntax depends on the simulation tool.

17 The $signed and $unsigned Functions

These functions specify how an expression is interpreted with respect to signedness.

17.1 Example

```
module signed_example;

    reg [7:0] data;

    initial begin
        data = 8'hFF;

        $display("Unsigned = %0d", $unsigned(data));
        $display("Signed   = %0d", $signed(data));

        $finish;
    end

endmodule
```

For an 8-bit value of `8'hFF`, the unsigned interpretation is 255 and the signed two's-complement interpretation is -1.

Signedness can affect comparisons, arithmetic, and expression evaluation. Use explicit sizing and signed declarations when necessary.

18 The $clog2 Function

The `$clog2` function returns the ceiling of the base-2 logarithm of a value.

It is commonly used to calculate address widths and counter widths.

18.1 Example

```
module clog2_example;

    parameter DEPTH = 16;
    localparam ADDR_WIDTH = $clog2(DEPTH);

    initial begin
        $display("Depth = %0d", DEPTH);
        $display("Address width = %0d", ADDR_WIDTH);
        $finish;
    end

endmodule
```

For a depth of 16, the address width is 4 bits.

For non-power-of-two depths, `$clog2` returns the ceiling required to represent the number of entries. Designs should still handle unused address encodings where necessary.

`$clog2` is available in Verilog-2005 and later language support.

19 System Tasks vs. User-Defined Tasks

System tasks are predefined by the language or simulation environment. User-defined tasks are written by the designer.

Feature	**System Task**	**User-Defined Task**
Definition	Predefined by the language or tool.	Defined by the designer.
Naming	Usually begins with `$`.	Uses a user-selected name.
Purpose	Simulation control, display, file I/O, and other built-in operations.	Reusable procedural operations.
Example	`$display`	`task print_data;`
Synthesis	Many are simulation-only.	Depends on the task contents and synthesis tool support.

20 Common Errors and Best Practices

Common errors include:

- Forgetting the dollar sign before a system task name.
- Using an incorrect format specifier.
- Confusing `$display` with `$write`.
- Expecting `$monitor` to print continuously when monitored values do not change.
- Forgetting to close a file after writing.
- Using a memory initialization file that cannot be found.
- Using SystemVerilog-only functions in a Verilog-only compilation mode.
- Including simulation-only tasks in synthesizable RTL without checking tool support.

Recommended practices include:

- Use `$display` for messages at specific simulation points.
- Use `$monitor` to track changes in selected signals.
- Use `$strobe` when final time-step values are needed.
- Use `$finish` to end testbench simulation cleanly.
- Use waveform dumping to debug complex signal interactions.
- Use descriptive messages and suitable format specifiers.
- Keep testbench code separate from synthesizable design RTL.

21 System Tasks Summary

Task or Function	Purpose
`$display`	Displays formatted output with a newline.
`$write`	Displays formatted output without an automatic newline.
`$monitor`	Prints values when monitored arguments change.
`$monitoron`	Enables monitor output.
`$monitoroff`	Disables monitor output.
`$strobe`	Displays values at the end of the current time step.
`$time`	Returns simulation time as an integer time value.
`$stime`	Returns a 32-bit unsigned simulation time value.
`$realtime`	Returns simulation time as a real number.
`$finish`	Terminates simulation.
`$stop`	Suspends or interrupts simulation for inspection.
`$random`	Generates a pseudo-random signed integer.
`$urandom`	Generates a pseudo-random unsigned value in SystemVerilog.
`$dumpfile`	Specifies the waveform dump file.
`$dumpvars`	Selects signals for waveform dumping.
`$readmemh`	Loads memory data from a hexadecimal file.
`$readmemb`	Loads memory data from a binary file.
`$fopen`	Opens a file.
`$fclose`	Closes a file.
`$fdisplay`	Writes formatted output to a file with a newline.
`$fwrite`	Writes formatted output to a file without an automatic newline.
`$test$plusargs`	Checks for a command-line plus argument.
`$value$plusargs`	Reads a value from a command-line plus argument.
`$signed`	Interprets an expression as signed.
`$unsigned`	Interprets an expression as unsigned.
`$clog2`	Returns the ceiling of the base-2 logarithm.

22 Summary

- System tasks and functions begin with the dollar sign.
- `$display`, `$write`, and `$monitor` are used to display simulation information.
- `$time` and `$realtime` provide simulation time information.
- `$finish` terminates simulation, while `$stop` supports interactive debugging.
- Waveform dumping tasks record signal activity for later analysis.
- Memory and file I/O tasks support testbench initialization and data logging.
- Some functions, including `$urandom`, are specific to SystemVerilog rather than classic Verilog.
- Many system tasks are intended for verification and are not synthesizable.

Key takeaway: System tasks and functions are essential tools for Verilog simulation, debugging, testbench development, and verification. Use them appropriately to observe behavior and diagnose design problems.